

Covert Channels: Storage Lab

Hiding Messages in File Permissions

2026-09-30

Table of contents

1	Setting Up an FTP Server	1
1.1	Installation	1
1.2	Logging In	2
1.3	Tweaking the FTP Server	2
1.4	Restarting the FTP Server	3
2	Storage Covert Channel	3
2.1	The 7-Bit Scheme	4
2.2	The 10-Bit Scheme	5

In the last lecture, you covered different ways that someone might set up and exploit a covert channel. In this lab, you will build one from scratch — a **storage covert channel** that hides a secret message inside the file permission bits of an FTP server.

To an ordinary observer browsing the FTP server, everything looks normal. But someone who knows the encoding scheme can read the hidden message simply by examining which permission bits are set on each file.

1 Setting Up an FTP Server

Tip

Depending on how far you went with a previous lab, you might not need to do this section at all. Skip ahead to Section 2 if your FTP server is already configured and running.

1.1 Installation

You will use **vsftpd** (Very Secure FTP Daemon), a lightweight and widely-used FTP server for Linux. Install it with the following commands:

```
sudo apt update          # refresh the package index
sudo apt install vsftpd # install vsftpd
```

Once installed, verify that vsftpd is running and listening on the network:

```
sudo netstat -alpn | grep ' LISTEN '
```

Tip

If the `netstat` command above doesn't work, you may need to install it first:

```
sudo apt install net-tools
```

Then re-run the `netstat | grep` command. Look for a line containing `vsftpd` to confirm the server is active.

1.2 Logging In

Try connecting to your own FTP server from the same machine:

```
ftp localhost
```

This will prompt for a username and password — use your system credentials. Once connected, explore what you have access to using standard FTP commands:

```
ls          # list files in the current directory
ls -lh      # list files with permissions and sizes
cd folder   # change into a directory
cd ..       # go up one level
Ctrl + D    # disconnect from the server
```

1.3 Tweaking the FTP Server

Make a few adjustments to improve the experience. Open the vsftpd configuration file:

```
sudo vim /etc/vsftpd.conf # or use nano, gedit, or any other editor
```

Find and adjust the following settings:

```
anonymous_enable=YES
local_enable=NO
ftpd_banner=My First FTP Server # customise this however you like
anon_root=/home/ftp            # create this directory if it doesn't exist
```

The `anon_root` setting controls which folder anonymous users land in when they connect. Create the directory first if needed:

```
sudo mkdir -p /home/ftp
```

1.4 Restarting the FTP Server

Restart the server to apply your changes, then log in again:

```
sudo service vsftpd restart
ftp localhost
```

This time you should be able to log in **anonymously** — username `anonymous`, blank password (just press Enter). You can also connect from another machine on the same network using the server's IP address:

```
ftp 138.47.x.x # replace with your server machine's IP address
```

Tip

If your FTP server is running inside a VM, the VM will have its own IP address distinct from the host machine. You can FTP into it from the host using that IP, provided the network adapter is set to bridged or NAT mode.

2 Storage Covert Channel

You will now use the FTP server as a storage covert channel — specifically, you will hide a message in the **permission bits** of the files on the server.

Linux file permissions consist of 10 characters: a file type indicator followed by three groups of read/write/execute flags for the owner, group, and other users respectively:

```
d  r w x   r w x   r w x
| \---\ / \---\ / \---\ /
| owner  group  other
\-- type (d=directory, -=file, l=link, ...)
```

The convention for our covert channel is simple: **any letter present = 1; a dash = 0**. This gives us up to 10 bits of information per file. There are two ways to exploit this, trading stealth for capacity.

2.1 The 7-Bit Scheme

Standard ASCII characters use 7 bits. We encode one character per file by writing the character's 7-bit value into the low 7 of the 9 permission bits and forcing the **top two bits** — **owner read and owner write** — **to zero**. (Prepending two zero bits to a 7-bit value is exactly what puts them there.) The owner *execute* bit is free and carries the high bit of the character, so a message file's owner column always reads --- or --x — never r or w.

That cleared owner read/write pair is also our marker. To keep the listing from looking too uniform, we intersperse **noise files** — directories, or files that *do* set an owner read or write bit. The receiver keeps only the regular files whose owner read and write bits are both clear, and skips everything else.

2.1.1 Worked Example: Encoding “Demigod”

First, convert each character to its 7-bit ASCII representation, then prepend two zero bits to fit the 9-bit permission string (three octal groups of three bits each):

Character	Decimal	7-bit binary	9-bit (00 prefix)	chmod value
D	68	1000100	00 100 010 0 → 001 000 100	104
e	101	1100101	00 110 010 1 → 001 100 101	145
m	109	1101101	00 110 110 1 → 001 101 101	155
i	105	1101001	00 110 100 1 → 001 101 001	151
g	103	1100111	00 110 011 1 → 001 100 111	147
o	111	1101111	00 110 111 1 → 001 101 111	157
d	100	1100100	00 110 010 0 → 001 100 100	144

The 9-bit representation groups neatly into three octal digits — exactly the format `chmod` expects. Create the files and set their permissions:

```
touch file1 && chmod 104 file1
touch file2 && chmod 145 file2
touch file3 && chmod 155 file3
touch file4 && chmod 151 file4
touch file5 && chmod 147 file5
touch file6 && chmod 157 file6
touch file7 && chmod 144 file7
```

The resulting directory listing will look like this. Note that no message file has any owner permission set:

```
---x---r-- 1 prof prof 0 Mar 27 16:15 file1
---xr--r-x 1 prof prof 0 Mar 27 16:15 file2
---xr-xr-x 1 prof prof 0 Mar 27 16:15 file3
---xr-x--- 1 prof prof 0 Mar 27 16:15 file4
---xr--rwx 1 prof prof 0 Mar 27 16:15 file5
```

```
---xr-xrwx 1 prof prof 0 Mar 27 16:15 file6
---xr--r-- 1 prof prof 0 Mar 27 16:15 file7
```

Now add noise files — each with at least one owner permission bit set — to make the listing look natural:

```
---x---r-- 1 prof prof 0 Mar 27 16:15 file1
d--xrw-r-- 1 prof prof 0 Mar 27 16:15 file1.5  <- noise (owner bit set)
---xr--r-x 1 prof prof 0 Mar 27 16:15 file2
---xr-xr-x 1 prof prof 0 Mar 27 16:15 file3
-r-xrwxrwx 1 prof prof 0 Mar 27 16:15 file3.5  <- noise
---xr-x--x 1 prof prof 0 Mar 27 16:15 file4
---xr--rwx 1 prof prof 0 Mar 27 16:15 file5
-rwx--xr-x 1 prof prof 0 Mar 27 16:15 file5.5  <- noise
---xr-xrwx 1 prof prof 0 Mar 27 16:15 file6
-rw-r---wx 1 prof prof 0 Mar 27 16:15 file6.5  <- noise
---xr--r-- 1 prof prof 0 Mar 27 16:15 file7
drwxr-xrw- 1 prof prof 0 Mar 27 16:15 file7.5  <- noise
```

SUCCESS — the covert channel is set up and the hidden message is embedded.

2.1.2 Decoding

To recover the message, a receiver scans each file, skips directories and any file that sets an owner read or write bit (noise), and reads the 9 permission bits of each surviving file as an ASCII character:

```
---x---r--  -> regular file, owner r/w clear -> 001000100 -> D
d--xrw-r--  -> directory (type 'd')          -> SKIP (noise)
---xr--r-x  -> regular file, owner r/w clear -> 001100101 -> e
...and so on
```

2.2 The 10-Bit Scheme

The 7-bit scheme wastes the first 3 permission bits. With a small change we can use all **10 bits** — squeezing more data out of each file.

The trade-off: without those “empty owner bits” to distinguish message files from noise files, we can no longer include noise. Every file in the listing carries part of the message, and the receiver processes them all.

To encode a message using 10 bits per file:

1. **Sort files alphabetically.** The receiver must process files in a consistent order; alphabetical filename order provides a natural, reproducible sequence.

2. **Flatten the bitstream.** Convert the entire covert message to binary, concatenate all bits into one long string, then split into groups of 10. Assign one group per file.
3. **Pad if necessary.** If the total bit count is not a multiple of 10, pad the end with zeros.
4. **Decode in 7-bit chunks.** The receiver collects all bits in order, splits them into 7-bit groups (standard ASCII), converts each group to its character, and reassembles the message.

2.2.1 Challenge: Decode This

Can you work out the hidden message in the listing below? Files are already sorted alphabetically. Collect the 10 bits from each file's permission string in order, then split the combined bitstream into 7-bit ASCII groups.

```
d---r--rwx 2 prof prof 4K Mar 27 20:57 0fd1b45f22e18b3
-r-xrw--w- 1 prof prof 0 Mar 27 20:57 17c455d90e49
-rw--w-r-x 1 prof prof 0 Mar 27 20:57 302289542768697c
-rw---x--- 1 prof prof 0 Mar 27 20:57 4bdf419390d83b860cec
--wxr-xrwx 1 prof prof 0 Mar 27 20:57 51451ddb647ff3566601f232
d-w---xr-- 2 prof prof 4K Mar 27 20:57 6e8dd5f0924ce30b35aeaed9
d-wxrw--w- 2 prof prof 4K Mar 27 20:57 70a8cbb30
dr--r-x-w- 2 prof prof 4K Mar 27 20:57 79bf30d265cbd436079e
-rwxrwx--x 1 prof prof 0 Mar 27 20:57 81052541de641ff1ed7ca40
d-w-----wx 2 prof prof 4K Mar 27 20:57 a8b18ffb171e161c753ab8d
-rw-rwxrw- 1 prof prof 0 Mar 27 20:57 c52eda933ff95be8f914eaf62
-r-x-----x 1 prof prof 0 Mar 27 20:57 daf9509999adb4f6e6b49c7e91
d---rwxr-- 2 prof prof 4K Mar 27 20:57 f35c8e8ed0fb8a609
--wxrw--w- 1 prof prof 0 Mar 27 20:57 f4ed4ab4e61c850de968
-rwxrwx-w- 1 prof prof 0 Mar 27 20:57 f59a77545fe6d10
---x----- 1 prof prof 0 Mar 27 20:57 fce47615d2
```

Answer

Decode each permission string to 10 bits and concatenate:

```
1000100111 0101110010 0110010101 0110001000 0011101111
0100011001 0110110111 0010110010 1111100110 1000001101
1011110010 1000001000 1111000111 1001001111 1010000100 0000
```

Group the combined bitstream into 7-bit ASCII chunks:

```
1000100 1110101 1100100 1100101 0101100 0100000 1110111 1101000
D           u           d           e           ,           [sp]           w           h

1100101 1110010 1100101 0100111 1110011 0100000 1101101 1111001
e           r           e           '           s           [sp]           m           y

0100000 1100011 1100001 1110010 0111111 0100001 [padding ignored]
[sp]           c           a           r           ?           !
```

The hidden message is: *Dude, where's my car?!*